# Docker Senior DevOps Engineer

Production Scenario Interview Preparation

**How to use:** Read the scenario, answer aloud, then use the bullet points to structure your response. Focus on **Check** → **Analyze** → **Mitigate** → **Fix** → **Monitor**.

## Scenario 1 — Production Container Memory / OOMKilled

**Scenario:** A production payment-service container runs with **--memory=2g** and **--cpus=1**. Users report intermittent 500 errors. **docker stats** shows about 1.95 GiB / 2 GiB and **docker inspect** shows **OOMKilled: true**. The container uses **--restart=unless-stopped**.

### Interview Answer

### What is happening?

- The container exceeded its 2 GB memory limit.
- The kernel/runtime killed the container because of memory exhaustion.
- The restart policy starts it again, so the problem can repeat.

### How I troubleshoot

- Run **docker stats payment-service** to observe memory over time.
- Run **docker inspect payment-service** and confirm OOMKilled and resource limits.
- Check **docker logs --tail 500 payment-service**.
- Correlate memory with traffic, latency, errors, CPU, and restart count.

### How I find the root cause

- Continuous memory growth under stable traffic suggests a memory leak.
- Memory rising only with traffic may indicate normal workload growth or undersizing.
- Check application metrics and profiling data.

### What I would do

- Do not permanently increase memory without understanding the cause.
- If production is impacted, temporarily increase the limit after checking host capacity.
- Fix the application/configuration if there is a leak or inefficiency.
- For Java, check heap, GC, heap dumps, GC logs, and JVM container settings.

### Prevention

- Monitor memory, OOMKilled events, restarts, CPU throttling, and application errors.
- Create alerts before the container reaches its memory limit.
- Load-test and size resource limits from real workload data.

**Easy memory flow:** Check → Analyze → Mitigate → Fix → Monitor

# Scenario 2 — Container Works Locally but Fails from Production Host

**Scenario:** A Node.js payment-service works inside the container and **curl localhost:8080/health** succeeds. In production, users get 502 errors and the host cannot reach **localhost:8080**. Container logs say the server started on port 8080.

## Interview Answer

## Likely cause

- Check the application binding first: it may be listening only on 127.0.0.1.
- For container networking, the application should normally listen on **0.0.0.0:8080**.
- Also verify Docker host-to-container port mapping.

## Commands

- Run **docker ps** and **docker port payment-service**.
- Run **docker inspect payment-service** to inspect ports and networks.
- Inside the container, verify the listener and test the health endpoint.
- Check Nginx/load balancer configuration because a 502 may come from the proxy layer.

## Fix

- Bind the application to **0.0.0.0** instead of 127.0.0.1 when required.
- Verify the run command uses **-p 8080:8080**.
- Check firewall/security rules if it works from the host but not externally.

## Prevention

- Add container/application health checks.
- Test image + port mapping in a production-like environment before release.
- Monitor load balancer errors, latency, and backend health.

**Easy memory flow:** Application → Container → Port Mapping → Network → Proxy/LB → Firewall

# Scenario 3 — MySQL Data Lost After Container Recreation

**Scenario:** MySQL runs in a Docker container without a mounted volume. The container is removed and a new MySQL container is created. The application starts, but customer/order data is gone.

### Interview Answer

### Why data was lost

- MySQL stores data under **/var/lib/mysql**.
- Without a persistent volume, that data lives in the container's writable layer.
- Removing the container removes that writable layer.
- A new container starts with a fresh database directory.

### How I verify

- Run **docker inspect mysql** and inspect the **Mounts** section.
- Run **docker volume ls** and check whether a persistent volume exists.

### Correct setup

- Create a named volume: **docker volume create mysql-data**.
- Mount it to **/var/lib/mysql**.
- A named volume survives container deletion and recreation.

### Important production point

- Do not mount the same MySQL data volume to multiple active MySQL instances.
- Use backups, restore testing, replication/HA, and preferably a managed database service for production.

**Easy memory flow:** Container = Temporary | Volume = Persistent | Backup = Recovery

# Scenario 4 — API Cannot Resolve Redis

**Scenario:** An API container cannot connect to Redis. Redis is running and **redis-cli ping** returns PONG, but the API gets **ECONNREFUSED** / cannot resolve **redis**. Investigation shows API is on **app-network** while Redis is on the default bridge network.

### Interview Answer

### Root cause

- The containers are on different Docker networks.
- Docker's embedded DNS/service-name resolution works within the relevant user-defined network.
- Therefore the API cannot resolve the Redis service name.

### Troubleshooting

- Run **docker network ls**.
- Run **docker network inspect <network>**.
- Inspect both containers and compare their network memberships.
- Test DNS and port connectivity from the API container.

### Fix

- If Redis is already running, connect it to the application network with **docker network connect app-network redis**.
- Use **redis:6379**, not a hardcoded container IP.

### Why service names?

- Container IPs can change when containers are recreated.
- Docker DNS resolves the service/container name to the current IP on the network.

### Prevention

- Define a custom network in Docker Compose.
- Use service names for communication.
- Add health checks and monitor Redis connection failures.

**Easy memory flow:** Same Network → DNS → Service Name → Port → Health Check → Monitoring

# Scenario 5 — Production Docker Host Disk Full

**Scenario:** A production host running 20 containers reaches 95% disk usage. **docker system df** shows many images, stopped containers, local volumes, and a large build cache. One container also has a very large log file.

## Interview Answer

## First response

- Treat it as a production capacity incident.
- Do not immediately run **docker system prune -a --volumes**.
- First identify what is consuming the disk and protect production data.

## Investigation

- Run **df -h** and host-level disk checks.
- Run **docker system df**.
- Review stopped containers, images, volumes, and build cache.
- Identify large Docker log files and application-generated files.

## Safe cleanup

- Remove confirmed-unused stopped containers.
- Remove confirmed-unused images or old versions that are not needed for rollback.
- Clean build cache when safe.
- Be extremely careful with volumes because they may contain database data.

## Logs

- Configure Docker log rotation, for example **max-size=100m** and **max-file=5**.
- Prefer centralized logging for production where appropriate.

## Prevention

- Alert at disk thresholds such as 80% warning and 90% critical.
- Implement image retention and cleanup policies.
- Monitor Docker storage and log growth.

**Easy memory flow:** Check → Protect → Clean → Recover → Monitor → Prevent

# Scenario 6 — New Deployment Causes High CPU

**Scenario:** Order-service v1.4 uses 30–40% CPU. After deploying v1.5, CPU reaches 95–100% while request traffic remains unchanged. Memory is stable and the container is not crashing.

## Interview Answer

## First assumption

- Do not immediately blame Docker.
- Because traffic is unchanged and CPU increased after the new version, treat the application release as the primary suspect.

## Troubleshooting

- Run **docker stats**, **docker top**, and **docker inspect**.
- Check application logs and APM/profiling data.
- Compare v1.4 and v1.5 for code, dependencies, retries, background jobs, database calls, and configuration changes.

## Production mitigation

- If v1.4 is known to be stable, prefer a controlled rollback rather than simply increasing CPU.
- Use rolling, canary, or blue/green deployment to avoid downtime.
- Monitor error rate, latency, CPU, memory, and business metrics during rollback.

## CPU increase?

- Increasing CPU can be a temporary mitigation if capacity is available.
- It should not be the permanent fix if the new version has inefficient code or a bug.

## Prevention

- Use load testing, canary deployments, health checks, resource limits, alerts, and automated rollback.

**Easy memory flow:** Detect → Compare → Investigate → Mitigate → Rollback → Fix → Prevent

# Scenario 7 — Critical Vulnerability in Production Docker Image

**Scenario:** Trivy reports 3 CRITICAL and 12 HIGH vulnerabilities in the production payment-service image. One critical issue comes from the base image. Security wants remediation, while the business cannot tolerate payment downtime.

## Interview Answer

## First response

- Do not immediately stop the production service without assessing the risk and having a remediation plan.
- Run Trivy and inspect the CVE, affected package, installed/fixed versions, exploitability, and runtime relevance.
- Do not simply ignore the issue because the application does not directly use the package.

## Fix the image

- Prefer a patched, supported base image.
- Consider a minimal/Alpine/distroless base only after compatibility testing.
- Use multi-stage builds to keep build tools out of the final image.
- Run as a non-root user and avoid unnecessary packages/capabilities.

## Avoid blind OS upgrades

- Do not blindly use **apt-get upgrade -y** as the security strategy.
- Prefer a maintained base image and intentional package updates.
- Rebuild, scan, and test the final image.

## Zero-downtime deployment

- Build v2.9, scan it, and run tests.
- Deploy using canary, rolling, or blue/green deployment.
- Start with a small percentage of traffic and monitor closely.
- Rollback immediately if health, latency, or payment success rates degrade.

## CI/CD prevention

- Run Trivy in Jenkins after the final image is built.
- Use policy gates to block CRITICAL vulnerabilities according to company policy.
- Do not promote or deploy images that fail the required security threshold.

**Easy memory flow:** Assess → Fix → Rebuild → Scan → Test → Canary → Monitor → Rollback